

module-17-images

Chapter 17: Manipulating Images | 24 programs in learning order

Source: *Automate the Boring Stuff with Python 1e*, Al Sweigart (CC-BY-NC-SA). Syllabus Ch17: Pillow colors, open/crop/paste/resize/rotate/pixels, logo project, drawing, photo finder

How to use this guide (beginner path)

1) Read Concepts once. 2) Follow the Program map for the big picture. 3) Work programs 01..N in order - each builds on the last. [shell] files: type every >>> line into IDLE yourself and compare with the # OUT: comments (errors shown are intentional demos). [script] files: save with the given name and run with F5; predict the output before you run.

Concepts in detail

Pillow opens images into objects: RGBA tuples (r,g,b,a) name colors; size/width/height describe; crop((l,t,r,b)) cuts; copy()+paste() composites; resize((w,h)) scales; rotate()/transpose() orient; getpixel()/putpixel() touch dots. Capstone: stamp a resized logo onto every photo in a tree and save. ImageDraw draws shapes/text; the photo-folder finder walks a disk for large images.

Key ideas to carry forward

- catIm.size -> (width, height); catIm.crop((335,345,565,560)) - box is left/top/right/bottom.
- logoIm.resize((w,h)); photo.paste(logo, (x,y), logo) keeps transparency with the mask.
- draw.text((x,y), 'Hi', font=fontObj) - load fonts via ImageFont.truetype().

Classic beginner mistakes

- ! Pasting without the mask loses transparency (white box around logo).
- ! Saving resized originals over inputs - write to a new folder.

Study order hint: 01-14 open/colors/crop/paste/resize/rotate/pixels, 15-19 logo capstone, 20-24 drawing + finder.

Program map - all 24 at a glance

```
01 01_colors_rgba_values.py
02 02_manipulating_images_pillow.py
03 03_manipulating_images_pillow_os.py
04 04_working_image_data.py
05 05_working_image_data_pil.py
06 06_cropping_images.py
07 07_copying_pasting_images.py
08 08_copying_pasting_images_faceim.py
09 09_copying_pasting_images_catimwidth.py
10 10_resizing_image.py
11 11_rotating_flipping_images.py
12 12_rotating_flipping_images_catim.py
13 13_rotating_flipping_images_catim_2.py
14 14_changing_individual_pixels.py
15 15_step_1_open.py
16 16_step_2_loop.py
17 17_step_3_resize.py
18 18_step_4_add.py
19 19_step_4_add_resizing.py
20 20_drawing_images.py
21 21_drawing_shapes.py
22 22_drawing_text.py
23 23_drawing_text_pil.py
24 24_identifying_photo_folders.py
```

Topic: Colors and RGBA Values

01/24 01_colors_rgba_values.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import ImageColor
ImageColor.getcolor('red', 'RGBA')
# OUT:      (255, 0, 0, 255)
ImageColor.getcolor('RED', 'RGBA')
# OUT:      (255, 0, 0, 255)
ImageColor.getcolor('Black', 'RGBA')
# OUT:      (0, 0, 0, 255)
ImageColor.getcolor('chocolate', 'RGBA')
# OUT:      (210, 105, 30, 255)
ImageColor.getcolor('CornflowerBlue', 'RGBA')
# OUT:      (100, 149, 237, 255)
```

Topic: Manipulating Images with Pillow

02/24 02_manipulating_images_pillow.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import Image
catIm = Image.open('zophie.png')
```

03/24 03_manipulating_images_pillow_os.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import os
```

```
os.chdir('C:\\folder_with_image_file')
```

Topic: Working with the Image Data Type

04/24 04_working_image_data.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import Image
catIm = Image.open('zophie.png')
catIm.size
# OUT:  (816, 1088)
width, height = catIm.size
width
# OUT:   816
height
# OUT:   1088
catIm.filename
# OUT:   'zophie.png'
catIm.format
# OUT:   'PNG'
catIm.format_description
# OUT:   'Portable network graphics'
catIm.save('zophie.jpg')
```

05/24 05_working_image_data_pil.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
from PIL import Image  
im = Image.new('RGBA', (100, 200), 'purple')  
im.save('purpleImage.png')  
im2 = Image.new('RGBA', (20, 20))  
im2.save('transparentImage.png')
```

Topic: Cropping Images

06/24 06_cropping_images.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
croppedIm = catIm.crop((335, 345, 565, 560))
```

```
croppedIm.save('cropped.png')
```

Topic: Copying and Pasting Images onto Other Images

07/24 07_copying_pasting_images.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
catIm = Image.open('zophie.png')
```

```
catCopyIm = catIm.copy()
```

08/24 08_copying_pasting_images_faceim.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
faceIm = catIm.crop((335, 345, 565, 560))  
faceIm.size  
# OUT: (230, 215)  
catCopyIm.paste(faceIm, (0, 0))  
catCopyIm.paste(faceIm, (400, 500))  
catCopyIm.save('pasted.png')
```

09/24 09_copying_pasting_images_catimwidth.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

catImWidth, catImHeight = catIm.size
faceImWidth, faceImHeight = faceIm.size
catCopyTwo = catIm.copy()
for left in range(0, catImWidth, faceImWidth):
# OUT:         for top in range(0, catImHeight, faceImHeight):
# OUT:             print(left, top)
# OUT:             catCopyTwo.paste(faceIm, (left, top))
# OUT:  0 0
# OUT:  0 215
# OUT:  0 430
# OUT:  0 645
# OUT:  0 860
# OUT:  0 1075
# OUT: 230 0
# OUT: 230 215
# OUT: --snip--
# OUT: 690 860
# OUT: 690 1075
catCopyTwo.save('tiled.png')
```

Topic: Resizing an Image

10/24 10_resizing_image.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
width, height = catIm.size
quartersizedIm = catIm.resize((int(width / 2), int(height / 2)))
quartersizedIm.save('quartersized.png')
svelteIm = catIm.resize((width, height + 300))
svelteIm.save('svelte.png')
```

Topic: Rotating and Flipping Images

11/24 11_rotating_flipping_images.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
catIm.rotate(90).save('rotated90.png')
catIm.rotate(180).save('rotated180.png')
catIm.rotate(270).save('rotated270.png')
```

12/24 12_rotating_flipping_images_catim.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
catIm.rotate(6).save('rotated6.png')
```

```
catIm.rotate(6, expand=True).save('rotated6_expanded.png')
```

13/24 13_rotating_flipping_images_catim_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
catIm.transpose(Image.FLIP_LEFT_RIGHT).save('horizontal_flip.png')
```

```
catIm.transpose(Image.FLIP_TOP_BOTTOM).save('vertical_flip.png')
```

Topic: Changing Individual Pixels

14/24 14_changing_individual_pixels.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

im = Image.new('RGBA', (100, 100))
im.getpixel((0, 0))
# OUT:      (0, 0, 0, 0)
for x in range(100):
# OUT:      for y in range(50):
# OUT:      im.putpixel((x, y), (210, 210, 210))
from PIL import ImageColor
for x in range(100):
# OUT:      for y in range(50, 100):
# OUT:      im.putpixel((x, y), ImageColor.getcolor('darkgray', 'RGBA'))
im.getpixel((0, 0))
# OUT:      (210, 210, 210, 255)
im.getpixel((0, 50))
# OUT:      (169, 169, 169, 255)
im.save('putPixel.png')
```

Topic: Step 1: Open the Logo Image

15/24 15_step_1_open.py [script]

Script: save and run; predict the output first, then compare.

```
#!/python3
# resizeAndAddLogo.py - Resizes all images in current working directory to fit
# in a 300x300 square, and adds catlogo.png to the lower-right corner.
import os
from PIL import Image

SQUARE_FIT_SIZE = 300
LOGO_FILENAME = 'catlogo.png'

logoIm = Image.open(LOGO_FILENAME)
logoWidth, logoHeight = logoIm.size

# TODO: Loop over all files in the working directory.

# TODO: Check if image needs to be resized.

# TODO: Calculate the new width and height to resize to.

# TODO: Resize the image.

# TODO: Add the logo.

# TODO: Save changes.
```

Topic: Step 2: Loop Over All Files and Open Images

16/24 16_step_2_loop.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# resizeAndAddLogo.py - Resizes all images in current working directory to fit
# in a 300x300 square, and adds catlogo.png to the lower-right corner.

import os
from PIL import Image

--snip--

os.makedirs('withLogo', exist_ok=True)
# Loop over all files in the working directory.
for filename in os.listdir('.'):
    if not (filename.endswith('.png') or filename.endswith('.jpg')) \
        or filename == LOGO_FILENAME:
        continue # skip non-image files and the logo file itself

    im = Image.open(filename)
    width, height = im.size
--snip--
```

Topic: Step 3: Resize the Images

17/24 17_step_3_resize.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# resizeAndAddLogo.py - Resizes all images in current working directory to fit
# in a 300x300 square, and adds catlogo.png to the lower-right corner.

import os
from PIL import Image

--snip--

# Check if image needs to be resized.
if width > SQUARE_FIT_SIZE and height > SQUARE_FIT_SIZE:
    # Calculate the new width and height to resize to.
    if width > height:
        height = int((SQUARE_FIT_SIZE / width) * height)
        width = SQUARE_FIT_SIZE
    else:
        width = int((SQUARE_FIT_SIZE / height) * width)
        height = SQUARE_FIT_SIZE

# Resize the image.
print('Resizing %s...' % (filename))
im = im.resize((width, height))

--snip--
```

Topic: Step 4: Add the Logo and Save the Changes

18/24 18_step_4_add.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# resizeAndAddLogo.py - Resizes all images in current working directory to fit
# in a 300x300 square, and adds catlogo.png to the lower-right corner.

import os
from PIL import Image

--snip--

# Check if image needs to be resized.
--snip--

# Add the logo.
print('Adding logo to %s...' % (filename))
im.paste(logoIm, (width - logoWidth, height - logoHeight), logoIm)

# Save changes.
im.save(os.path.join('withLogo', filename))
```

19/24 19_step_4_add_resizing.py [script]

Script: save and run; predict the output first, then compare.

```
Resizing zophie.png...
```

```
Adding logo to zophie.png...
```

Topic: Drawing on Images

20/24 20_drawing_images.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import Image, ImageDraw
im = Image.new('RGBA', (200, 200), 'white')
draw = ImageDraw.Draw(im)
```

Topic: Drawing Shapes

21/24 21_drawing_shapes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import Image, ImageDraw
im = Image.new('RGBA', (200, 200), 'white')
draw = ImageDraw.Draw(im)
draw.line([(0, 0), (199, 0), (199, 199), (0, 199), (0, 0)], fill='black')
draw.rectangle((20, 30, 60, 60), fill='blue')
draw.ellipse((120, 30, 160, 60), fill='red')
draw.polygon(((57, 87), (79, 62), (94, 85), (120, 90), (103, 113)),
# OUT:      fill='brown')
for i in range(100, 200, 10):
# OUT:      draw.line([(i, 0), (200, i - 100)], fill='green')

im.save('drawing.png')
```

Topic: Drawing Text

22/24 22_drawing_text.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from PIL import ImageFont
```

23/24 23_drawing_text_pil.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
from PIL import Image, ImageDraw, ImageFont  
import os  
im = Image.new('RGBA', (200, 200), 'white')  
draw = ImageDraw.Draw(im)  
draw.text((20, 150), 'Hello', fill='purple')  
fontsFolder = 'FONT_FOLDER' # e.g. 'Library/Fonts'  
arialFont = ImageFont.truetype(os.path.join(fontsFolder, 'arial.ttf'), 32)  
draw.text((100, 150), 'Howdy', fill='gray', font=arialFont)  
im.save('text.png')
```

Topic: Identifying Photo Folders on the Hard Drive

24/24 24_identifying_photo_folders.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3 #
Import modules and write comments to describe this program.

for foldername, subfolders, filenames in os.walk('C:\\\\'):
    numPhotoFiles = 0
    numNonPhotoFiles = 0
    for filename in filenames:
        # Check if file extension isn't .png or .jpg.
        if TODO:
            numNonPhotoFiles += 1
            continue    # skip to next filename

        # Open image file using Pillow.

        # Check if width & height are larger than 500.
        if TODO:
            # Image is large enough to be considered a photo.
            numPhotoFiles += 1
        else:
            # Image is too small to be a photo.
            numNonPhotoFiles += 1

    # If more than half of files were photos,
    # print the absolute path of the folder.
    if TODO:
        print(TODO)
```